LAPORAN PRAKTIKUM

INTERNET OF THINGS II

Modul 4: Komunikasi Asinkron dan WebSockets



Disusun Oleh :

Dhimaz dhewananda (25782037)

TRI 3B

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

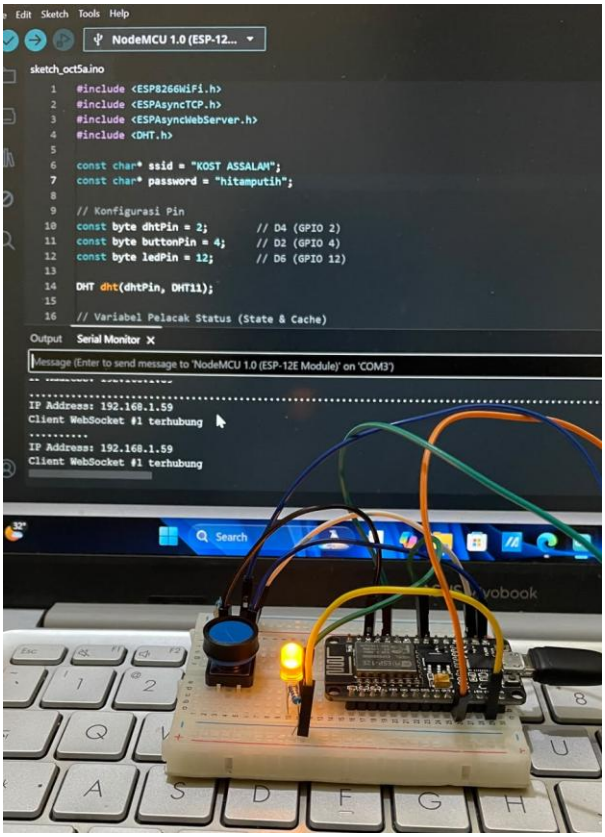
TEKNOLOGI REKAYASA INTERNET

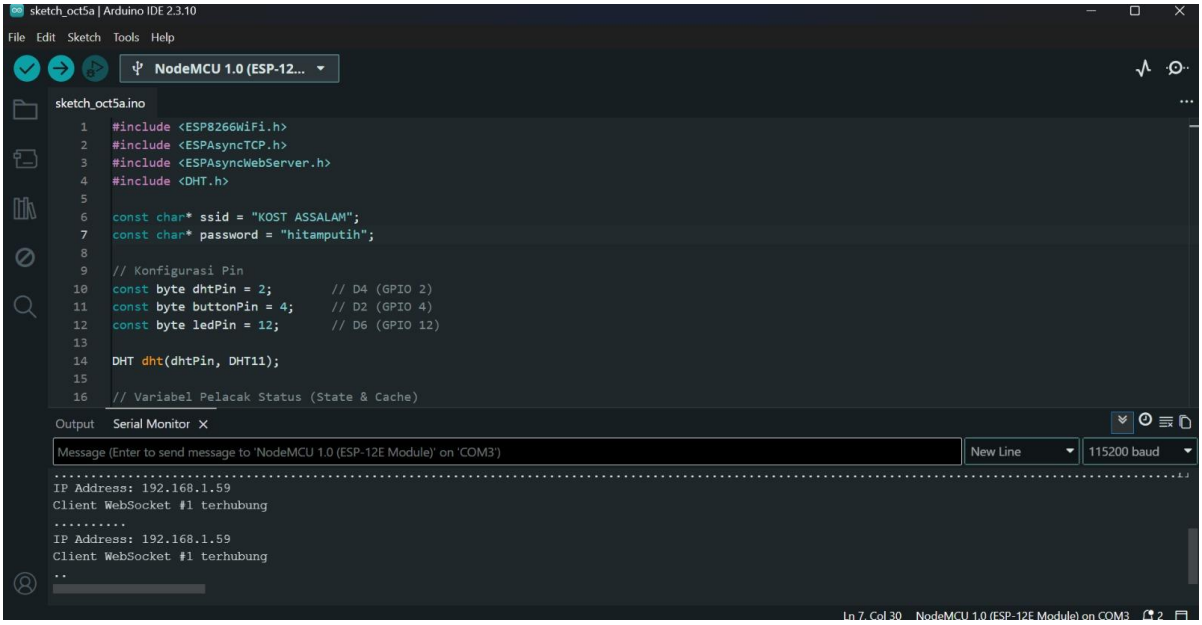
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

4.5 Praktikum





Praktikum Modul 4 ini berhasil mengimplementasikan komunikasi *real-time* dua arah (*full-duplex*) antara Web Dashboard dan NodeMCU ESP8266 menggunakan protokol WebSockets.

4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

  WebSockets cuma membuka jalur koneksi satu kali saja (*persistent connection*). Data dikirim dalam paket teks ringkas (JSON), dan JavaScript langsung mengubah isi teks di halaman tanpa unduh ulang kode HTML. Makanya hemat *bandwidth* dan bebas kedipan (*no flickering*), beda dengan *Meta Refresh* yang mengunduh ulang seluruh halaman dari awal.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

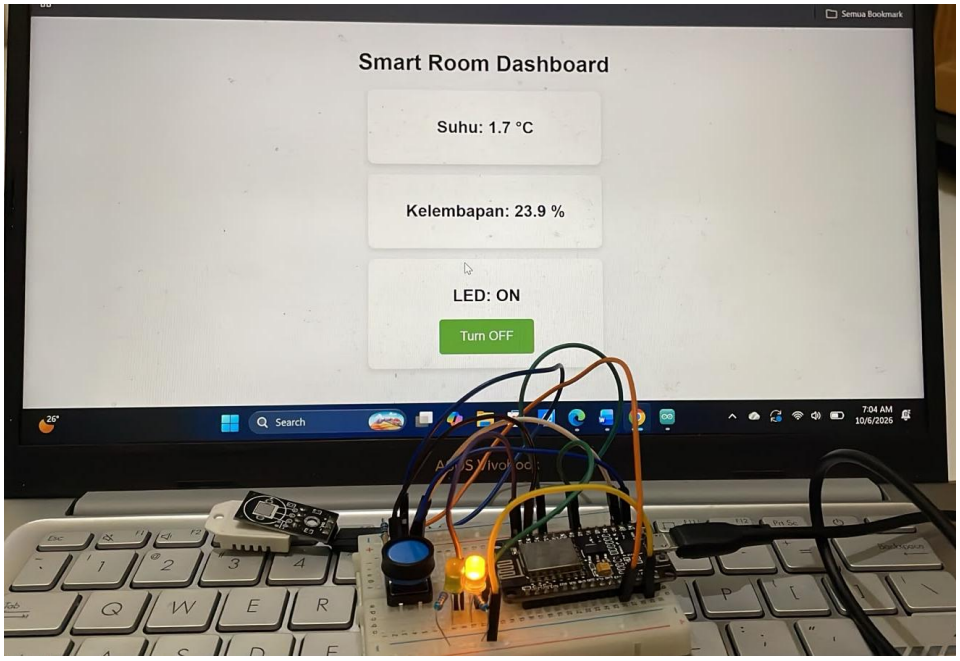
  Penggunaan delay(3000) membuat CPU NodeMCU berhenti total (*freeze*) selama 3 detik. Dampaknya:
- Perintah tombol di web jadi macet/terlambat (*lag*).
- Ditekan tombol fisik tidak akan terbaca.
- Jalur WebSocket bisa terputus atau NodeMCU mengalami *restart* sendiri (*Watchdog Timer Reset*).

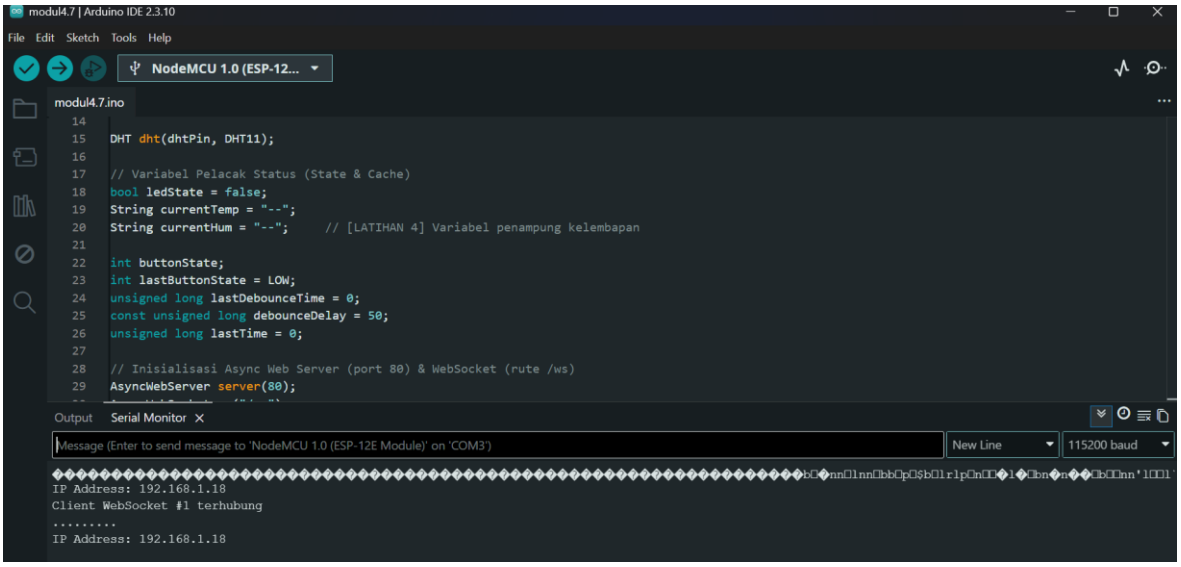
- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?
  (Modifikasi Front-End)
  ID elemen HTML yang diubah oleh JavaScript:

1. **tempValue** → Menampilkan angka suhu.
2. **ledStatus** → Menampilkan status ON / OFF.
3. **toggleBtn** → Mengubah teks tombol & warna CSS.

- **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
  i. Sisi Back-End (C++): Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
  ii. Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen

humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!





Penambahan fitur pembacaan kelembapan dari sensor DHT11 terbukti sukses dimuat pada kartu baru "Kelembapan: 23.9 %". Angka kelembapan dan suhu (1.7 °C) pada layar ter-update secara otomatis setiap 3 detik secara *real-time* tanpa perlu kita *refresh* halamannya. Selain itu, fitur tombol kontrol LED juga siap digunakan untuk mematikan atau menyalakan LED fisik secara langsung lewat tampilan web.